Métodos Numéricos

Prof. Jonatha Costa

2026

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

MN aplicados à Engenharia

Objetivo da aula

- Apresentar conteúdo de Ajuste de Curvas
  - Interpolação e extrapolação
  - Regressão Linear por Mínimos Quadrados
  - Linearização de Equações não lineares
  - Polinômio de Lagrange e de Newton
  - Spline linear, quadrática e cúbica

MN aplicados à Engenharia

Considerações basilares de engenharia e modelagem matemática

1. Em Engenharia, grandezas físicas são frequentemente obtidas por meio de **medições experimentais**, realizadas por sensores e sistemas de aquisição de dados;
2. As medições são armazenadas como um conjunto de **dados discretos**, constituindo os chamados **dados ou pontos experimentais**;
3. A representação discreta fornece apenas uma quantidade finita de informações sobre o fenômeno observado, podendo não reproduzir integralmente o comportamento contínuo da grandeza;
4. A partir dos pontos experimentais, busca-se obter uma **função matemática** capaz de representar, de forma adequada, o comportamento observado;
5. Essa representação é obtida por meio do **ajuste de curvas**, no qual os parâmetros de uma função são determinados de modo a minimizar uma medida de erro entre o modelo e os dados experimentais;
6. Dependendo do problema, podem ser utilizadas diferentes funções de aproximação, incluindo **polinômios de diferentes ordens**.

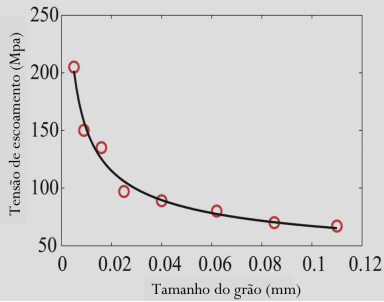
MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Definição

- **Interpolação**: procedimento utilizado para **estimar valores entre pontos experimentais conhecidos**, dentro do intervalo dos dados disponíveis.
- **Extrapolação**: procedimento utilizado para **estimar valores fora do intervalo** dos dados experimentais, utilizando o comportamento identificado no modelo.
- **Ajuste de curvas**: procedimento utilizado para determinar os **parâmetros de uma função matemática** que represente adequadamente um conjunto de dados, segundo um determinado **critério de erro**.

Figura 1: Tamanho de grão × tensão



Fonte: GILAT,(2008)

Perceba na Figura 1 os pontos discretos de uma medição que pode ser expressa pela Equação 1.

$$\sigma = Cd^m \tag{1}$$

em que:

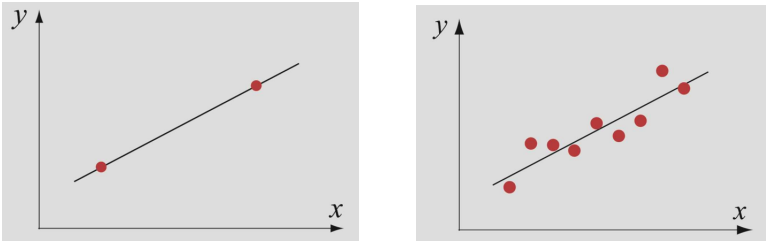
- σ - Tensão de escoamento
- C - é uma constante
- d - é o tamanho do grão
- m - é o grau da equação

MN aplicados à Engenharia

Ajuste de curvas – Polinômio de 1º grau:

- É uma das formas mais simples de representar matematicamente a relação entre duas variáveis a partir de dados experimentais;
- O modelo é definido por $y = a_1x + a_0$, em que a_1 representa a inclinação da reta e a_0 o intercepto com o eixo y;
- Considere a Figura 2.
  - Para **dois pontos**, existe uma única reta que passa exatamente pelos pontos experimentais;
  - Para **vários pontos**, os dados podem não estar sobre uma única reta. Nesse caso, a_0 e a_1 são determinados de modo que a reta forneça a melhor aproximação possível aos dados, segundo um determinado critério de erro.

Figura 2: Curvas



(a) Dois pontos (b) Vários pontos

Fonte: GILAT,(2008)

MN aplicados à Engenharia

Ajuste de Curvas e Interpolação

Qualidade do ajuste de curvas

- Para avaliar o quanto uma função representa um conjunto de dados experimentais, é necessário definir uma **medida de erro** entre os valores observados e os valores estimados pelo modelo;
- O **resíduo** representa a diferença entre o valor experimental y_i e o valor estimado pela função $f(x_i)$;
- Para um conjunto de n pontos experimentais, cada ponto possui um resíduo associado;
- Quanto menores forem os resíduos, menor tende a ser a diferença entre o modelo matemático e os dados experimentais.

Valor experimental	**Valor estimado**
y_i	$\hat{y}_i = f(x_i)$

Resíduo do i-ésimo elemento de y pode ser expresso pela Equação 2.

$$\boxed{r_i = y_i - f(x_i)} \tag{2}$$

MN aplicados à Engenharia

Ajuste de Curvas e Interpolação

Melhor ajuste – Método dos mínimos quadrados

- Note que os resíduos podem assumir valores positivos ou negativos. Portanto, a simples soma dos resíduos pode resultar em cancelamentos;
- Para evitar esse efeito, utiliza-se a **soma dos quadrados dos resíduos** como medida global do erro;
- Para um polinômio de primeiro grau: $f(x) = a_1 x + a_0$, o erro quadrático global é definido pela Equação 3.

$$E(a_0, a_1) = \sum_{i=1}^{n} r_i^2 = \sum_{i=1}^{n} [y_i - (a_1 x_i + a_0)]^2 \tag{3}$$

Critério de mínimos quadrados expresso na Equação 4.

$$\min_{a_0, a_1} E(a_0, a_1) = \min_{a_0, a_1} \sum_{i=1}^{n} [y_i - (a_1 x_i + a_0)]^2 \tag{4}$$

O melhor ajuste é aquele que **minimiza a soma dos quadrados dos resíduos**.

Como $r_i^2 \geq 0$, resíduos positivos e negativos não se cancelam.

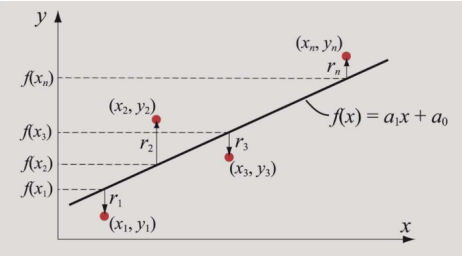
MN aplicados à Engenharia

Ajuste de Curvas e Interpolação

Melhor ajuste – Método dos mínimos quadrados

- Considere a Figura 3. Note que os resíduos podem assumir valores positivos ou negativos. A simples soma dos resíduos pode, portanto, produzir cancelamentos;
- Para evitar esse efeito, utiliza-se a **soma dos quadrados dos resíduos** como medida global do erro;
- Para um polinômio de primeiro grau: $f(x) = a_1 x + a_0$, o erro quadrático global é expresso na Equação 5.

Figura 3: Polinômio de grau 1



Fonte: Gilat,(2008).

Erro quadrático global

$$E(a_0, a_1) = \sum_{i=1}^{n} r_i^2 \qquad (5)$$

Reescrita como: $\sum_{i=1}^{n} [y_i - (a_1 x_i + a_0)]^2$

$r_i^2 \geq 0 \Rightarrow$ não há cancelamento entre resíduos positivos e negativos.

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

MN aplicados à Engenharia

Regressão Linear pelo método dos Mínimos Quadrados

- A minimização da soma dos quadrados dos resíduos conduz a um sistema de duas equações, obtido pela anulação das derivadas parciais de E em relação a a_0 e a_1.
- Portanto, o melhor ajuste é obtido quando o erro global, definido pela equação a seguir, é mínimo:

$$\min_{a_0,a_1} E(a_0,a_1) = \min_{a_0,a_1} \sum_{i=1}^{n} [y_i - (a_1 x_i + a_0)]^2$$

- Derivando o erro E em função dos coeficientes a_0 e a_1, tem-se:

$$\frac{\partial E}{\partial a_0} = -2 \sum_{i=1}^{n} \Big[y_i - a_i x_i - a_0 \Big] = 0$$

$$\frac{\partial E}{\partial a_1} = -2 \sum_{i=1}^{n} \Big[y_i - a_i x_i - a_0 \Big] x_i = 0$$

MN aplicados à Engenharia

Regressão Linear pelo método dos Mínimos Quadrados

- Resolvendo as derivadas, tem-se que:

$$a_1 = \frac{n\sum_{i=1}^{n} x_i y_i - \left(\sum_{i=1}^{n} x_i\right)\left(\sum_{i=1}^{n} y_i\right)}{n\sum_{i=1}^{n} x_i^2 - \left(\sum_{i=1}^{n} x_i\right)^2}$$

$$a_0 = \frac{\left(\sum_{i=1}^{n} x_i^2\right)\left(\sum_{i=1}^{n} y_i\right) - \left(\sum_{i=1}^{n} x_i y_i\right)\left(\sum_{i=1}^{n} x_i\right)}{n\sum_{i=1}^{n} x_i^2 - \left(\sum_{i=1}^{n} x_i\right)^2}$$

- Por conveniência pode-se expressar o somatório como S_k:

$$\boxed{S_x = \sum_{i=1}^{n} x_i}, \quad \boxed{S_y = \sum_{i=1}^{n} y_i}, \quad \boxed{S_{xy} = \sum_{i=1}^{n} x_i y_i}, \quad \boxed{S_{xx} = \sum_{i=1}^{n} x_i^2}.$$

Portanto, os termos a_0 e a_1 de $f(x) = a_1 x + a_0$, são: $\boxed{a_1 = \frac{nS_{xy} - S_x S_y}{nS_{xx} - (S_x)^2}}$, $\boxed{a_0 = \frac{S_{xy} S_y - S_{xy} S_x}{nS_{xx} - (S_x)^2}}$

MN aplicados à Engenharia

Regressão Linear em Python

- O código a seguir utiliza a estruturação anteriormente detalhada.

Cálculo dos coeficientes

```
import numpy as np
import matplotlib.pyplot as plt
x = np.arange(1, 9)
y = np.array([2.1, 3.9, 4.2, 6.1,
7.5, 8.3, 8.9, 10.4])

def reglin(x, y):
    if len(x) != len(y):
            return print('Falha!')
    n = len(x)
    Sx, Sy = sum(x), sum(y)
    Sxy, Sxx = sum(x*y), sum(x**2)
    a1 = (n*Sxy - Sx*Sy)/(n*Sxx - Sx**2)
    a0 = (Sxx*Sy - Sxy*Sx)/(n*Sxx - Sx**2)
    return a1, a0
a1, a0 = np.round(reglin(x, y), 4)
print(f"p(x) = {a1}*x + {a0}")
```

Visualização da regressão

```
def graf_reglin(x, y):
    a1, a0 = reglin(x, y)

    print("Coef. angular:", a1)
    print("Coef. linear:", a0)

    plt.scatter(x, y, label="Dados")
    plt.plot(x, a0 + a1*x, 'r-',
    label="Regressao linear")
    plt.xlabel("x")
    plt.ylabel("y")
    plt.legend()
    plt.grid()
    plt.show()

graf_reglin(x, y)
```

MN aplicados à Engenharia

Regressão Linear em Python

- O código a seguir utiliza o comando `np.polyfit(x,y,1)` da biblioteca `NumPy` para calcular os coeficientes da reta de regressão linear pelo método dos mínimos quadrados.

Código Python

```
import numpy as np                                  # Biblioteca de operacoes numericas
import matplotlib.pyplot as plt                     # Biblioteca de graficos
x = np.arange(1, 9)                                 # Dados de entrada
y = np.array([2.1, 3.9, 4.2, 6.1,                   # Dados observados
7.5, 8.3, 8.9, 10.4])
a1, a0 = np.polyfit(x, y, 1)                        # Calcula os coeficientes a1 e a0
print("Coef. angular:", a1)                         # Exibe o coeficiente angular
print("Coef. linear:", a0)                          # Exibe o coeficiente linear
plt.scatter(x, y, label="Dados")                    # Traca os dados observados
plt.plot(x, a0 + a1*x, 'r-',                        # Traca a reta ajustada
label="Regressao linear")
plt.xlabel("x")                                     # Define o rotulo do eixo x
plt.ylabel("y")                                     # Define o rotulo do eixo y
plt.legend()                                        # Exibe a legenda do grafico
plt.grid()                                          # Adiciona a grade ao grafico
plt.show()                                          # Exibe o grafico
```

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Exercícios de Fixação — Regressão Linear (I)

1. Considere os dados experimentais a seguir:

$$X = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$$

$$Y = \{2.1, 3.9, 5.2, 7.1, 8.0, 10.2, 11.1, 13.2, 14.1, 16.2, 17.0, 19.1\}$$

   a) Determine os coeficientes a_0 e a_1 pelo método dos mínimos quadrados e obtenha a equação da reta ajustada.

   b) Implemente a regressão linear em Python utilizando `np.polyfit()`. Apresente os coeficientes calculados e construa o gráfico dos dados experimentais e da reta ajustada.

   c) Utilize a reta obtida para estimar y quando $x = 10.5$ e $x = 15$. Classifique essa estimativa como interpolação ou extrapolação e justifique sua resposta.

MN aplicados à Engenharia

Ajuste de Curvas e Interpolação

Exercícios de Fixação — Regressão Linear (II)

2. **Ajuste linear de dados experimentais**

   Em um experimento de Engenharia, foram obtidos os seguintes dados, relacionando a temperatura x (°C) e uma grandeza física medida y:

   $$X = \{10, 20, 30, 40, 50, 60\}$$

   $$Y = \{12.1, 18.3, 25.8, 31.2, 39.1, 44.7\}$$

   a) Implemente em Python o ajuste linear $\hat{y} = a_1 x + a_0$ utilizando `np.polyfit()`.
   b) Construa um gráfico contendo os dados experimentais e a reta ajustada.
   c) Calcule o coeficiente de determinação

3. **Interpolação e extrapolação**

   Utilizando o modelo ajustado no Exercício 1:

   a) Estime y para $x = 35$ e $x = 75$.
   b) Classifique cada estimativa como interpolação ou extrapolação, justificando sua resposta.
   c) Discuta por que a confiabilidade das estimativas pode diminuir fora do intervalo experimental.

MN aplicados à Engenharia

Ajuste de Curvas e Interpolação

Exercícios de Fixação — Regressão Linear (III)

4. **Comparação entre ajustes lineares**

Um ensaio de laboratório produziu os seguintes resultados para a relação entre o tempo x (s) e a distância percorrida y (m):

$$X = \{1, 2, 3, 4, 5, 6, 7\}$$

$$Y = \{2.8, 5.1, 7.9, 8.8, 11.2, 13.1, 16.0\}$$

Implemente um programa em Python que:

a) Determine a reta de regressão linear pelo método dos mínimos quadrados utilizando `np.polyfit()`.
b) Calcule os valores estimados $\hat{y}_i$ e os resíduos $e_i = y_i - \hat{y}_i$.
c) Construa um gráfico dos dados e da reta ajustada e outro gráfico dos resíduos em função de x.
d) Calcule o erro quadrático médio (EQM) e o coeficiente de determinação R^2.
e) Analise o gráfico dos resíduos e discuta se o modelo linear é adequado para representar os dados experimentais.

MN aplicados à Engenharia

Curiosidade: Métodos de Regressão Linear

Há somente um método de regressão linear?

MN aplicados à Engenharia

Curiosidade: Métodos de Regressão Linear

Há somente um método de regressão linear?

- **Mínimos Quadrados Ordinários (MQO):** minimiza os resíduos quadráticos. $\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$
- **Mínimos Quadrados Ponderados (MQP):** atribui pesos às observações. $\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} w_i (y_i - \hat{y}_i)^2$
- **Mínimos Quadrados Generalizados (MQG):** considera a covariância dos erros. $\min_{\boldsymbol{\beta}} (\mathbf{y} - \mathbf{X}\beta)^T \boldsymbol{\Sigma}^{-1} (\mathbf{y} - \mathbf{X}\beta)$
- **Regressão Linear Múltipla:** utiliza várias variáveis explicativas. $\hat{y} = a_0 + a_1 x_1 + \cdots + a_p x_p$
- **Regressão Regularizada (Ridge e Lasso):** penaliza os coeficientes para controlar a complexidade.

  Ridge: $J + \lambda \sum_{j=1}^{p} a_j^2$ Lasso: $J + \lambda \sum_{j=1}^{p} |a_j|$
- **Gradiente Descendente:** atualiza os coeficientes iterativamente. $\boldsymbol{\beta}^{(k+1)} = \boldsymbol{\beta}^{(k)} - \alpha \nabla J(\boldsymbol{\beta}^{(k)})$
- **Regressão Robusta:** reduz a influência de valores atípicos. $\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} \rho(y_i - \hat{y}_i)$

Pesquise sobre os demais métodos distintos acima, visando comparar a performance de implementação computacional!

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

Métricas de Erro: MAE, MSE e RMSE

Definição matemática

MAE

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

MSE

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

RMSE

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$$

Implementação em Python

```python
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error
def mae(y_true, y_pred):
    return mean_absolute_error(y_true, y_pred)
def mse(y_true, y_pred):
    return mean_squared_error(y_true, y_pred)
def rmse(y_true, y_pred):
    return np.sqrt(mse(y_true, y_pred))
```

- **MAE:** média dos módulos dos erros.
- **MSE:** média dos erros ao quadrado.
- **RMSE:** raiz do MSE, na unidade da variável prevista.

Métricas Percentuais e de Ajuste

Definição matemática

MAPE

$$\text{MAPE} = \frac{100}{N} \sum_{i=1}^{N} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Coeficiente R^2

$$R^2 = 1 - \frac{\sum_{i=1}^{N} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{N} (y_i - \bar{y})^2}$$

Bias

$$\text{Bias} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)$$

Implementação em Python

```
import numpy as np
from sklearn.metrics import r2_score
def mape(y_true, y_pred):
    return np.mean(np.abs((y_true - y_pred) / y_true)) * 100
def r2(y_true, y_pred):
    return r2_score(y_true, y_pred)
def bias(y_true, y_pred):
    return np.mean(y_pred - y_true)
```

- **MAPE:** erro absoluto percentual médio.
- R^2**:** proporção da variabilidade explicada pelo modelo, em relação à média dos valores reais.
- **Bias:** viés médio; positivo indica superestimação e negativo indica subestimação.

Métricas Complementares e Resultados

Definição matemática

MSLE – Erro Logarítmico Quadrático Médio

$$\text{MSLE} = \frac{1}{N}\sum_{i=1}^{N}[\ln(1+y_i) - \ln(1+\hat{y}_i)]^2$$

WMSE – Erro Quadrático Médio Ponderado

$$\text{WMSE} = \frac{\sum_{i=1}^{N} w_i(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N} w_i}$$

Implementação em Python

```python
import numpy as np
from sklearn.metrics import mean_squared_error
def msle(y_true, y_pred):
    return mean_squared_error(np.log1p(y_true), np.log1p(y_pred))
def wmse(y_true, y_pred, weights=None):
    if weights is None:
    weights = np.array([0.1, 0.2, 0.3, 0.4])
    return np.sum(weights * (y_true-y_pred)**2) / np.sum(weights)
if __name__ == "__main__":
    y_true = np.array([3.0, -0.5, 2.0, 7.0])
    y_pred = np.array([2.5,  0.0, 2.0, 8.0])
    print(f"MSLE: {msle(y_true, y_pred):.4f}")
    print(f"WMSE: {wmse(y_true, y_pred):.4f}")
```

- **MSLE:** avalia erros no domínio logarítmico, reduzindo a influência de diferenças absolutas grandes.
- **WMSE:** permite atribuir maior importância a observações específicas por meio dos pesos w_i.

Exercícios de erros

Implemente os erros acima nos exercícios de regressão linear.

1. Exercício I - slide 15.
2. Exercício II - slide 16.
3. Exercício III - slide 17.

Atenção

Implemente as métricas de erro em modo procedural e utilizando operações matemáticas básicas, **sem utilizar funções nativas**. Em seguida, compare os resultados com os obtidos pelas funções nativas correspondentes, verificando a consistência e a precisão das implementações.

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

MN aplicados à Engenharia

Linearização de Equações

- Muitas situações na ciência e na engenharia mostram que a relação entre as grandezas envolvidas **não é linear**;
- Entretanto, funções não-lineares que podem ser escritas em uma forma tal que possibilite a determinação dos coeficientes que levam ao melhor ajuste com o emprego do método da regressão linear por mínimos quadrados;
  - $y = b \cdot x^m$
- Para que a regressão linear possa ser utilizada, a equação não-linear de duas variáveis deve ser modificada de tal forma que a nova equação seja linear com termos contendo as variáveis originais. Dessa forma, para a equação acima tem-se:

$$\boxed{y = b \cdot x^m \rightarrow ln(y) = ln(b \cdot x^m) = m \cdot ln(x) + ln(b)}$$

  - equação linear em termos de $ln(x)$ tal que a equação está na forma $Y = a_1 X + a_0$, em que $Y = ln(y)$, $a_1 = m$,$X = ln(x)$ e $a_0 = ln(b)$.
  - Isso significa que uma regressão linear por mínimos quadrados pode ser usada para fazer com que uma equação na forma $y = b \cdot x^m$ se ajuste a um conjunto de pontos x_i, y_i.

Linearização de Equações Não Lineares

Na Tabela 1 são apresentadas algumas equações não lineares expressas em formas lineares no formato $Y = a_1 X + a_0$.

Tabela 1: Equações não lineares na forma linear.

Equação não linear	Forma linear	Relação com $Y = a_1 X + a_0$	Valores para a regressão linear por mínimos quadrados	Gráficos onde os dados medidos parecem se ajustar a uma linha reta
$y = bx^m$	$\ln(y) = m\ln(x) + \ln(b)$	$Y = \ln(y), \quad X = \ln(x)$ $a_1 = m, \quad a_0 = \ln(b)$	$\ln(x_i)$ e $\ln(y_i)$	Gráfico y vs. x em eixos x e y logarítmicos. Gráfico $\ln(y)$ vs. $\ln(x)$ em eixos lineares.
$y = be^{mx}$	$\ln(y) = mx + \ln(b)$	$Y = \ln(y), \quad X = x$ $a_1 = m, \quad a_0 = \ln(b)$	x_i e $\ln(y_i)$	Gráfico y vs. x com eixo x linear e eixo y logarítmico. Gráfico $\ln(y)$ vs. x em eixos lineares.
$y = b10^{mx}$	$\log(y) = mx + \log(b)$	$Y = \log(y), \quad X = x$ $a_1 = m, \quad a_0 = \log(b)$	x_i e $\log(y_i)$	Gráfico y vs. x com eixo x linear e eixo y logarítmico. Gráfico $\log(y)$ vs. x em eixos lineares.
$y = \dfrac{1}{mx + b}$	$\dfrac{1}{y} = mx + b$	$Y = \dfrac{1}{y}, \quad X = x$ $a_1 = m, \quad a_0 = b$	x_i e $\dfrac{1}{y_i}$	Gráfico $1/y$ vs. x em eixos lineares.
$y = \dfrac{mx}{b + x}$	$\dfrac{1}{y} = \dfrac{b}{m}\dfrac{1}{x} + \dfrac{1}{m}$	$Y = \dfrac{1}{y}, \quad X = \dfrac{1}{x}$ $a_1 = \dfrac{b}{m}, \quad a_0 = \dfrac{1}{m}$	$\dfrac{1}{x_i}$ e $\dfrac{1}{y_i}$	Gráfico $1/y$ vs. $1/x$ em eixos lineares.

Nota: x_i e y_i representam os dados medidos. Para as linearizações logarítmicas, exige-se $x_i > 0$ e $y_i > 0$.

MN Aplicados à Engenharia

Linearização de Equações Não Lineares

A escolha de um modelo não linear para o ajuste de curvas deve considerar o comportamento dos dados experimentais e as restrições matemáticas da função. Alguns critérios importantes são:

1. **Análise gráfica:** representar os dados em diferentes escalas (linear, semilogarítmica ou logarítmica) para verificar se alguma transformação produz aproximadamente uma relação linear.

2. **Modelo exponencial:** para $y = be^{mx}$ ou $y = b10^{mx}$, com $b \neq 0$, a função não passa pela origem. A linearização logarítmica exige $y > 0$ (ou $y < 0$, quando se utiliza $\ln|y|$ e $b < 0$).

3. **Modelo logarítmico:** para $y = a + b\ln(x)$, o domínio exige $x > 0$.

4. **Modelo de potência:** para $y = bx^m$, com $m > 0$, tem-se $y = 0$ quando $x = 0$. A linearização por logaritmos exige $x > 0$ e $y > 0$, considerando $b > 0$.

5. **Modelo inverso:** para $y = \dfrac{1}{ax + b}$, exige-se $ax + b \neq 0$. Além disso, a transformação $Y = 1/y$ requer $y \neq 0$.

6. **Adequação do modelo:** verificar se a função representa o fenômeno físico estudado e se os parâmetros estimados são coerentes com o problema de Engenharia.

MN Aplicados à Engenharia

Exercícios de Fixação – Linearização com Python (I)

1. **Ajuste de uma função de potência** Considere os dados experimentais:

$$x = [1, 2, 3, 4, 5], \qquad y = [2.1, 8.3, 18.5, 32.2, 50.4].$$

Implemente um programa em Python que:

- transforme os dados usando $X = \ln(x)$ e $Y = \ln(y)$;
- determine os coeficientes a_1 e a_0 por mínimos quadrados;
- calcule os parâmetros $m = a_1$ e $b = e^{a_0}$;
- apresente o modelo ajustado $y = bx^m$;
- construa os gráficos dos dados experimentais e do modelo ajustado.

2. **Ajuste de uma função exponencial**
Considere os dados:

$$x = [0, 1, 2, 3, 4], \qquad y = [2.0, 3.3, 5.5, 9.0, 14.8].$$

Desenvolva um programa que:

- linearize o modelo $y = be^{mx}$;
- estime m e b por regressão linear;
- reconstrua a equação exponencial;
- compare os dados medidos com os valores previstos;
- apresente o gráfico de $\ln(y)$ em função de x.

MN Aplicados à Engenharia

Exercícios de Fixação – Linearização com Python (II)

3. **Linearização de uma exponencial de base 10** Considere os dados:

$$x = [0, 1, 2, 3, 4], \qquad y = [3.0, 6.0, 12.0, 24.0, 48.0].$$

Implemente um programa para:

- linearizar $y = b10^{mx}$ usando $\log_{10}(y)$;
- estimar os parâmetros m e b;
- comparar o modelo ajustado com os dados experimentais;
- verificar a relação entre o coeficiente angular e a taxa de crescimento exponencial.

4. **Comparação entre modelos não lineares** Considere os modelos

$$y = \frac{1}{mx + b}, \qquad y = \frac{mx}{b + x}.$$

Escolha ou gere conjuntos de dados compatíveis com cada modelo e implemente um programa que:

- aplique as transformações lineares apropriadas;
- estime os parâmetros por mínimos quadrados;
- reconstrua os modelos originais;
- compare os valores previstos com os dados;
- discuta as restrições e limitações de cada transformação.

5. **Análise dos resultados**
Para os modelos implementados, calcule o erro quadrático médio (EQM) na escala original e na escala transformada. Compare os resultados e explique por que minimizar o erro na escala transformada não garante o menor erro na escala original.

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

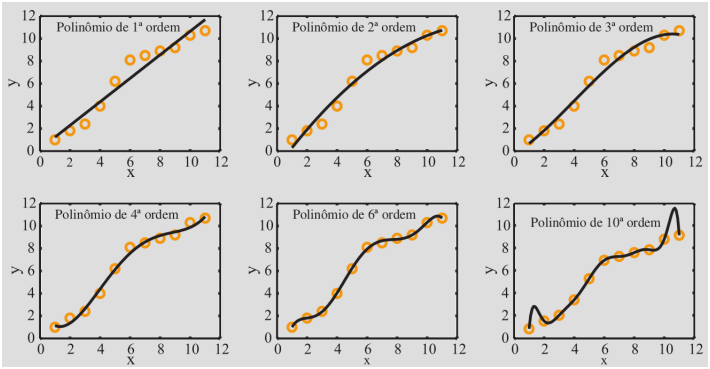
MN aplicados à Engenharia

Ajuste de curvas com polinômios de ordem superior

Considere a Figura 4. Note que:

1. Um polinômio de 1ª ordem representa uma reta; um de 2ª ordem, uma parábola; e um de 3ª ordem pode apresentar um ponto de inflexão, dependendo de seus coeficientes;
2. Quanto maior a ordem do polinômio, maior sua flexibilidade para representar variações e curvaturas nos dados;
3. A escolha do polinômio que proporciona o *melhor ajuste* depende da natureza e da origem dos dados, da aplicação e do objetivo da modelagem;
4. Observe as discrepâncias entre os dados observados e as curvas ajustadas nas regiões centrais e nas extremidades de cada gráfico da figura.

Figura 4: Polinômio de grau 'n'



Fonte: GILAT,(2008)

MN aplicados à Engenharia

Ajuste de curvas com polinômios quadráticos e de ordem superior

Considerações

1. É possível construir um polinômio interpolador que passe exatamente por n pontos distintos, desde que seu grau seja, no máximo, $(n-1)$;
2. Quando os dados apresentam incertezas ou imprecisões, a utilização de polinômios de ordem elevada pode ser desnecessário e até inadequado;
3. Embora um polinômio interpolador de grau elevado reproduza exatamente os pontos utilizados na construção do modelo, ele pode apresentar oscilações significativas entre esses pontos;
4. Nessas condições, o polinômio pode não representar adequadamente o comportamento dos dados, prejudicando a confiabilidade das estimativas por interpolação e, principalmente, por extrapolação.

MN aplicados à Engenharia

Regressão polinomial

- Considere um polinômio de grau m, definido por:

$$f(x) = \sum_{i=0}^{m} a_i x^i = a_0 + a_1 x + \cdots + a_{m-1} x^{m-1} + a_m x^m.$$

- Para um conjunto de n pontos observados (x_i, y_i), o ajuste polinomial de grau m consiste em determinar os coeficientes $a_0, a_1, \ldots, a_m$ que minimizam a soma dos quadrados dos resíduos:

$$E = \sum_{i=1}^{n} \left[y_i - \left(a_0 + a_1 x_i + \cdots + a_{m-1} x_i^{m-1} + a_m x_i^m \right) \right]^2.$$

- Ao calcular as derivadas parciais de E em relação aos coeficientes $a_0, a_1, \ldots, a_m$ e igualá-las a zero, obtém-se um sistema de $m+1$ equações normais. A solução desse sistema determina os coeficientes do polinômio que minimiza o erro quadrático total.

- Para n pontos com abscissas distintas, quando $m = n-1$, existe um polinômio interpolador que reproduz exatamente todos os pontos, resultando em $E = 0$. Para $m < n-1$, o ajuste por mínimos quadrados busca o polinômio que melhor representa a tendência dos dados, minimizando a soma dos quadrados dos resíduos, sem exigir que ele passe por todos os pontos.

MN Aplicados à Engenharia

Considere o ajuste de um polinômio de **grau** $m = 2$, definido por: $f(x) = a_0 + a_1 x + a_2 x^2$.
O método dos mínimos quadrados consiste em determinar os coeficientes a_0, a_1 e a_2 que minimizam a soma dos quadrados dos resíduos:

$$E(a_0, a_1, a_2) = \sum_{i=1}^{n} \left[y_i - (a_0 + a_1 x_i + a_2 x_i^2) \right]^2 .$$

- Para determinar os coeficientes do polinômio ajustado, calculam-se as derivadas parciais de E em relação a a_0, a_1 e a_2, igualando-as a zero.
- As condições de primeira ordem para a minimização são:

$$\frac{\partial E}{\partial a_0} = -2 \sum_{i=1}^{n} \left[y_i - (a_0 + a_1 x_i + a_2 x_i^2) \right] = 0,$$

$$\frac{\partial E}{\partial a_1} = -2 \sum_{i=1}^{n} x_i \left[y_i - (a_0 + a_1 x_i + a_2 x_i^2) \right] = 0,$$

$$\frac{\partial E}{\partial a_2} = -2 \sum_{i=1}^{n} x_i^2 \left[y_i - (a_0 + a_1 x_i + a_2 x_i^2) \right] = 0.$$

Resultado

As três condições resultam em um sistema de três equações normais, cuja solução fornece os coeficientes do polinômio que minimiza o erro quadrático total.

MN Aplicados à Engenharia

Ajuste de Curvas e Interpolação

A partir das condições de minimização do erro, obtém-se o seguinte **sistema de equações normais**:

$$\begin{cases} na_0 + \left(\sum_{i=1}^{n} x_i\right) a_1 + \left(\sum_{i=1}^{n} x_i^2\right) a_2 = \sum_{i=1}^{n} y_i, \\ \left(\sum_{i=1}^{n} x_i\right) a_0 + \left(\sum_{i=1}^{n} x_i^2\right) a_1 + \left(\sum_{i=1}^{n} x_i^3\right) a_2 = \sum_{i=1}^{n} x_i y_i, \\ \left(\sum_{i=1}^{n} x_i^2\right) a_0 + \left(\sum_{i=1}^{n} x_i^3\right) a_1 + \left(\sum_{i=1}^{n} x_i^4\right) a_2 = \sum_{i=1}^{n} x_i^2 y_i. \end{cases}$$

- A solução do sistema de equações normais fornece os coeficientes a_0, a_1 e a_2 do polinômio quadrático

$$f(x) = a_2 x^2 + a_1 x + a_0,$$

que proporciona o melhor ajuste aos n pontos observados (x_i, y_i), segundo o critério de minimização da soma dos quadrados dos resíduos.

- Esse procedimento pode ser generalizado para polinômios de grau m, resultando em um sistema de $m+1$ equações normais e $m+1$ incógnitas, correspondente aos coeficientes $a_0, a_1, \ldots, a_m$.

MN Aplicados à Engenharia

Formulação Matricial do Ajuste Polinomial

As equações normais podem ser organizadas na forma matricial:

$$\underbrace{\begin{bmatrix} n & \sum x_i & \sum x_i^2 \\ \sum x_i & \sum x_i^2 & \sum x_i^3 \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix}}_{\boldsymbol{a}} = \underbrace{\begin{bmatrix} \sum y_i \\ \sum x_i y_i \\ \sum x_i^2 y_i \end{bmatrix}}_{\boldsymbol{b}}, \qquad i = 1, \ldots, n.$$

- O sistema assume a forma compacta $A\boldsymbol{a} = \boldsymbol{b}$.
  Se A for **não singular**, sua solução pode ser expressa formalmente por $\boldsymbol{a} = A^{-1}\boldsymbol{b}$.
- Para fins de implementação computacional, observe:
  - Os elementos de A seguem um padrão baseado nas somas das potências de x_i; os elementos de $\boldsymbol{b}$ envolvem os produtos entre y_i e as potências de x_i.
  - O vetor de coeficientes é organizado em ordem crescente de grau: $\boldsymbol{a} = [a_0, a_1, a_2]^T$.
  - Essa convenção corresponde à representação

$$p(x) = a_0 + a_1 x + \cdots + a_m x^m.$$

- Caso se utilize a ordem decrescente, $[a_m, \ldots, a_1, a_0]$, a implementação deverá respeitar essa convenção.

MN Aplicados à Engenharia

Implementação Computacional: Regressão Polinomial

Código Python

```python
import numpy as np                                                    # Operacoes numericas
import matplotlib.pyplot as plt                                       # Graficos
x  = np.arange(1, 9)                                                  # Valores de entrada
y  = np.array([2.1, 3.9, 4.2, 6.1, 7.5, 8.3, 8.9, 10.4])              # Dados observados
m  = 2                                                                # Grau do polinomio
a  = np.polyfit(x, y, m)                                              # Estima os coeficientes
xx = np.linspace(x.min(), x.max(), 200)                               # Pontos para a curva
yy = np.polyval(a,xx)                                                 # Avalia o polinomio
print("Coeficientes:", a)                                             # Exibe os coeficientes
plt.scatter(x, y, label="Dados observados")                           # Plota os dados
plt.plot(xx, yy, "r-", label="Regressao polinomial")                  # Plota o ajuste
plt.xlabel("x")                                                       # Rotulo do eixo x
plt.ylabel("y")                                                       # Rotulo do eixo y
plt.legend()                                                          # Exibe a legenda
plt.grid(True)                                                        # Ativa a grade
plt.show()                                                            # Exibe o grafico
```

MN Aplicados à Engenharia

Exercício Computacional: Regressão Polinomial

Considere os dados experimentais abaixo e utilize Python para realizar o ajuste polinomial pelo método dos mínimos quadrados.

Dados experimentais

$$x = [1, 2, 3, 4, 5, 6, 7, 8]$$

$$y = [2.1, 3.9, 4.2, 6.1, 7.5, 8.3, 8.9, 10.4]$$

1. **Implementação:** escreva um programa em Python para ajustar um polinômio de grau $m = 2$ aos dados experimentais, utilizando a função `polyfit` da biblioteca NumPy.

2. **Análise dos coeficientes:** apresente os coeficientes calculados pelo programa e escreva a equação do polinômio ajustado na forma
$$p(x) = a_0 + a_1 x + a_2 x^2.$$

3. **Visualização e comparação:** construa um gráfico contendo os pontos experimentais e a curva ajustada. Repita o procedimento para os graus $m = 1, 2, 3, 4$ e $m = 5$ e compare os resultados, identificando as diferenças entre os ajustes. Apresente uma tabela com resultados comparativos, utilizando como critérios os erros MAE, RMSE e R2.

MN Aplicados à Engenharia

Exercício Computacional: Avaliação da Regressão Polinomial

Considere os dados experimentais abaixo e avalie a qualidade dos modelos polinomiais obtidos pelo método dos mínimos quadrados.

Dados experimentais ($n = 20$)

$$x = [1.2, 1.8, 2.5, 3.1, 3.7, 4.2, 4.9, 5.4, 6.0, 6.6, 7.1, 7.8, 8.3, 8.9, 9.5, 10.2, 10.8, 11.4, 12.1, 12.8]$$

$$y = [3.4, 4.1, 4.8, 5.2, 6.0, 6.3, 7.1, 7.4, 8.2, 8.5, 9.1, 9.8, 10.0, 10.7, 11.1, 12.0, 12.3, 13.1, 13.5, 14.2]$$

1. **Previsão:** ajuste um polinômio de grau $m = 2$ e estime y para $x = 7.0$.
2. **Análise dos resíduos:** calcule $e_i = y_i - p(x_i)$ e construa um gráfico dos resíduos em função de x. Verifique se os resíduos apresentam padrões sistemáticos.
3. **Comparação dos modelos:** calcule a soma dos quadrados dos resíduos (SSE) para os polinômios de graus 1 ao 10. Compare os resultados e discuta os efeitos do aumento da complexidade do modelo. Apresente os resultados numa tabela, utilizando erros R2, RMSE e MAE.

MN Aplicados à Engenharia

Regressão Polinomial – Considerações Finais

- Na regressão polinomial, os coeficientes são determinados pelo método dos mínimos quadrados, buscando minimizar a soma dos quadrados dos resíduos entre os valores observados e os valores estimados pelo modelo.
- Diferentemente da interpolação, não se exige que a curva ajustada passe por todos os pontos experimentais. O objetivo é obter uma representação matemática que descreva a **tendência** geral dos dados.
- Quando se deseja reproduzir exatamente um conjunto de pontos conhecidos, torna-se necessário empregar métodos de interpolação.

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

MN Aplicados à Engenharia

Interpolação Polinomial

Motivação

A interpolação polinomial consiste na determinação de uma função polinomial que reproduza exatamente um conjunto de pontos conhecidos, permitindo estimar valores em posições intermediárias do domínio de interesse.

- Considere um conjunto de n pontos distintos:

$$(x_1, y_1),\ (x_2, y_2),\ \ldots,\ (x_n, y_n).$$

- O objetivo é determinar um polinômio $P_{n-1}(x)$, de grau máximo $n-1$, que satisfaça as condições:

$$\boxed{P_{n-1}(x_i) = y_i, \qquad i = 1, 2, \ldots, n.}$$

- Entre as principais formulações para a construção do polinômio interpolador, destacam-se:
  - **Método de Lagrange:** utiliza polinômios básicos construídos a partir das abscissas conhecidas.
  - **Método de Newton:** utiliza diferenças divididas para determinar a representação do polinômio interpolador.

MN Aplicados à Engenharia

Interpolação Polinomial – Lagrange

- Considere dois pontos distintos, (x_1, y_1) e (x_2, y_2), com $x_1 \neq x_2$. O polinômio interpolador de primeiro grau pode ser expresso na forma:

$$P_1(x) = a_1(x - x_2) + a_2(x - x_1).$$

- Impondo as condições de interpolação $P_1(x_1) = y_1$ e $P_1(x_2) = y_2$, obtêm-se:

$$y_1 = a_1(x_1 - x_2) \Rightarrow a_1 = \frac{y_1}{x_1 - x_2},$$

$$y_2 = a_2(x_2 - x_1) \Rightarrow a_2 = \frac{y_2}{x_2 - x_1}.$$

- Substituindo os coeficientes na expressão inicial, resulta:

$$\boxed{P_1(x) = \frac{x - x_2}{x_1 - x_2} y_1 + \frac{x - x_1}{x_2 - x_1} y_2}$$

- Cada parcela representa uma função básica de Lagrange multiplicada pelo respectivo valor observado.

MN aplicados à Engenharia

Polinômio de Lagrange

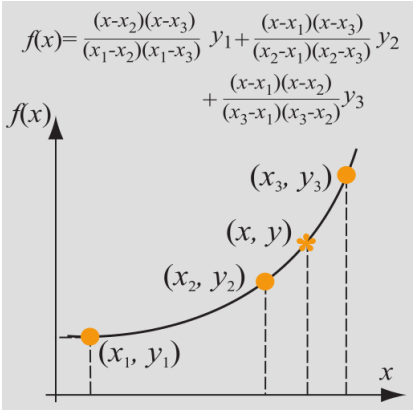
Considere os pontos (x_1, y_1), (x_2, y_2) e (x_3, y_3), na Figura 5.

- O polinômio interpolador de grau máximo dois é dado por:

$$P_2(x) = \underbrace{\frac{(x-x_2)(x-x_3)}{(x_1-x_2)(x_1-x_3)}}_{L_1(x)} y_1 + \underbrace{\frac{(x-x_1)(x-x_3)}{(x_2-x_1)(x_2-x_3)}}_{L_2(x)} y_2 + \underbrace{\frac{(x-x_1)(x-x_2)}{(x_3-x_1)(x_3-x_2)}}_{L_3(x)} y_3.$$

- As funções $L_1(x)$, $L_2(x)$ e $L_3(x)$ são os polinômios básicos de Lagrange, responsáveis pela construção da função interpoladora.

Figura 5: Lagrange de 2ª ordem



Fonte: GILAT,(2008)

MN Aplicados à Engenharia

Interpolação Polinomial – Formulação Geral

- Para um conjunto de n pontos observados, (x_i, y_i), com abscissas distintas, o polinômio interpolador de grau máximo $n-1$ pode ser escrito pela Equação 6:

$$P_{n-1}(x) = \sum_{i=1}^{n} y_i L_i(x) \tag{6}$$

- As funções básicas de Lagrange são definidas pela Equação 7:

$$L_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j} \tag{7}$$

- Essas funções satisfazem a propriedade fundamental:

$$L_i(x_j) = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

- **Propriedades e considerações:**
  1. As abscissas dos pontos não precisam ser equidistantes.
  2. O polinômio interpola exatamente todos os pontos utilizados em sua construção.
  3. A inclusão de novos pontos exige a reconstrução da expressão interpoladora na formulação convencional.
  4. Para conjuntos extensos de dados, o cálculo direto pode apresentar custo computacional elevado.

MN Aplicados à Engenharia

Polinômio de Lagrange – Implementação em Python

Exercício resolvido

Um ensaio experimental forneceu os seguintes valores para a relação entre uma variável independente x e uma variável dependente y:

x_i	0	1	2	3
y_i	1	3	2	5

Determine o polinômio interpolador de Lagrange e estime o valor de $f(1.5)$.

Implementação em Python

```
import numpy as np
x = np.array([0, 1, 2, 3], dtype=float)
y = np.array([1, 3, 2, 5], dtype=float)

def lagrange(x, y, valor):
    n = len(x)
    resultado = 0.0
    for i in range(n):
        L = 1.0
        for j in range(n):
            if i != j:
                L *= (valor - x[j]) / (x[i] - x[j])
        resultado += y[i] * L
    return resultado

print(f"f(1.5) = {lagrange(x, y, 1.5):.4f}")
```

Resultado

$$\boxed{f(1.5) = 2.438}$$

Exercícios — Interpolação de Lagrange

1. **Interpolação polinomial de grau 4**
   Considere os vetores:
   $$\mathbf{x} = [0, 1, 2, 3, 4],$$
   $$\mathbf{y} = [1, 3, 2, 5, 4].$$
   Determine o polinômio interpolador $P_4(x)$ utilizando Lagrange e calcule $P_4(2{,}5)$.

2. **Interpolação polinomial de grau 5**
   Considere os dados experimentais:
   $$\mathbf{x} = [0, 1, 2, 3, 4, 5],$$
   $$\mathbf{y} = [2, 4, 3, 6, 5, 8].$$
   Construa o polinômio interpolador e estime o valor correspondente a $x = 2{,}5$.

3. **Implementação computacional**
   Implemente uma função em Python para calcular o polinômio interpolador de Lagrange. Utilize:
   $$\mathbf{x} = [1, 2, 3, 4, 5, 6, 7],$$
   $$\mathbf{y} = [2, 5, 4, 8, 7, 10, 9].$$
   Determine as estimativas em $x = 2{,}5$ e $x = 5{,}5$.

Exercícios — Aplicações de Lagrange

4. **Estimativa de temperatura**
Um sistema de monitoramento registrou os seguintes dados:

$$\mathbf{t} = [0, 2, 4, 6, 8, 10],$$
$$\mathbf{T} = [20, 24, 29, 31, 30, 27].$$

Utilize Lagrange para estimar a temperatura em $t = 3$, $t = 5$ e $t = 9$ segundos.

5. **Comparação entre métodos**
Considere os vetores:

$$\mathbf{x} = [0, 1, 2, 3, 4, 5],$$
$$\mathbf{y} = [1, 2, 5, 10, 17, 26].$$

Implemente os métodos de Lagrange e Newton em Python. Compare os resultados obtidos para $x = 2{,}5$ e $x = 4{,}5$.

6. **Análise computacional**
Considere os dados:

$$\mathbf{x} = [0, 1, 2, 3, 4, 5, 6, 7, 8],$$
$$\mathbf{y} = [3, 5, 8, 12, 17, 23, 30, 38, 47].$$

Implemente o método de Lagrange em Python e:

- Calcule $P_8(x)$ para $x = 2{,}5$, $4{,}5$ e $7{,}5$;
- Verifique a reprodução dos pontos originais;
- Analise o efeito do aumento do número de pontos sobre o grau do polinômio interpolador.

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

MN Aplicados à Engenharia

Ajuste de Curvas e Interpolação

Polinômio Interpolador de Newton

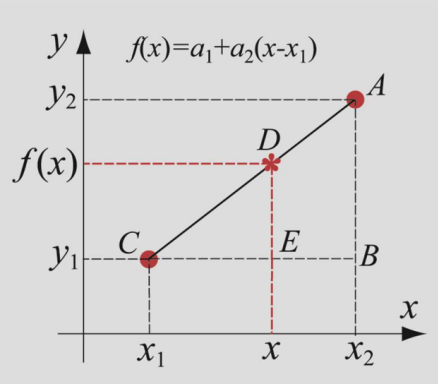
- Método alternativo ao polinômio interpolador de Lagrange, baseado no cálculo de **diferenças divididas**.
- Principais características:
  - Não exige a resolução de um sistema de equações lineares para determinar os coeficientes;
  - Os pontos não precisam estar ordenados, embora sua ordenação facilite a organização dos cálculos;
  - Os coeficientes calculados podem ser reutilizados para avaliar o polinômio em diferentes valores de x;
  - Permite a inclusão de novos pontos sem recalcular os coeficientes anteriormente determinados. Basta calcular os novos coeficientes;
  - Apresenta uma estrutura incremental, especialmente útil quando novos dados são incorporados ao conjunto original.
- **Forma geral:** o polinômio interpolador de Newton que passa por n pontos distintos possui grau máximo $n-1$, conforme expresso na Equação 8.

$$P_{n-1}(x) = a_0 + a_1(x - x_1) + a_2(x - x_1)(x - x_2) + \cdots + a_{n-1}\prod_{j=1}^{n-1}(x - x_j). \tag{8}$$

MN Aplicados à Engenharia

Polinômio de Newton de primeira ordem

Figura 6: Polinômio de Newton.



Fonte: Gilat (2008).

Considere os pontos (x_1, y_1) e (x_2, y_2) da Figura 6.
O polinômio interpolador de primeira ordem é:

$$P_1(x) = a_1 + a_2(x - x_1).$$

Aplicando as condições de interpolação:

$$P_1(x_1) = y_1 \quad \Rightarrow \quad a_1 = y_1.$$

$$P_1(x_2) = y_2 \quad \Rightarrow \quad a_2 = \frac{y_2 - y_1}{x_2 - x_1}.$$

Portanto:

$$\boxed{P_1(x) = y_1 + \frac{y_2 - y_1}{x_2 - x_1}(x - x_1)}$$

MN Aplicados à Engenharia

Polinômios de Newton de segunda e terceira ordens

- Para três pontos distintos, (x_1, y_1), (x_2, y_2) e (x_3, y_3), o polinômio de segunda ordem é:

$$P_2(x) = a_1 + a_2(x - x_1) + a_3(x - x_1)(x - x_2).$$

- Os dois primeiros coeficientes são os mesmos determinados na interpolação linear:

$$a_1 = y_1, \qquad a_2 = \frac{y_2 - y_1}{x_2 - x_1}.$$

- O terceiro coeficiente é calculado por:

$$\boxed{a_3 = \frac{\dfrac{y_3 - y_2}{x_3 - x_2} - \dfrac{y_2 - y_1}{x_2 - x_1}}{x_3 - x_1}}$$

- Para quatro pontos, acrescenta-se um novo termo:

$$P_3(x) = P_2(x) + a_4(x - x_1)(x - x_2)(x - x_3).$$

- O coeficiente adicional é:

$$\boxed{a_4 = \frac{f[x_4, x_3, x_2] - f[x_3, x_2, x_1]}{x_4 - x_1}}$$

MN Aplicados à Engenharia

Polinômio de Newton — Diferenças Divididas

Para determinar os coeficientes de ordem superior, utiliza-se o conceito de **diferenças divididas**.

- **Diferença dividida de primeira ordem:**

$$f[x_i, x_{i-1}] = \frac{y_i - y_{i-1}}{x_i - x_{i-1}}$$

- **Diferença dividida de segunda ordem:**

$$f[x_i, x_{i-1}, x_{i-2}] = \frac{f[x_i, x_{i-1}] - f[x_{i-1}, x_{i-2}]}{x_i - x_{i-2}}$$

- **Diferença dividida de terceira ordem:**

$$f[x_i, x_{i-1}, x_{i-2}, x_{i-3}] = \frac{f[x_i, x_{i-1}, x_{i-2}] - f[x_{i-1}, x_{i-2}, x_{i-3}]}{x_i - x_{i-3}}$$

- **Relação com os coeficientes de Newton:**

$$a_1 = y_1, a_2 = f[x_2, x_1], a_3 = f[x_3, x_2, x_1], a_4 = f[x_4, x_3, x_2, x_1].$$

MN Aplicados à Engenharia

Polinômio de Newton — Forma Geral

Diferenças divididas A k-ésima diferença dividida é determinada recursivamente pela diferença entre duas diferenças divididas de ordem anterior.

$$f[x_k, \ldots, x_1] = \frac{f[x_k, \ldots, x_2] - f[x_{k-1}, \ldots, x_1]}{x_k - x_1}$$

Os primeiros coeficientes são:

$$\begin{aligned} a_1 &= y_1, \\ a_2 &= f[x_2, x_1], \\ a_3 &= f[x_3, x_2, x_1], \\ &\vdots \\ a_n &= f[x_n, \ldots, x_1]. \end{aligned}$$

Forma geral do polinômio interpolador Para n pontos distintos, o polinômio de Newton pode ser escrito como:

$$\begin{aligned} P_{n-1}(x) = y_1 &\\ &+ f[x_2, x_1](x - x_1) \\ &+ f[x_3, x_2, x_1](x - x_1)(x - x_2) \\ &+ \cdots \\ &+ f[x_n, \ldots, x_1] \prod_{j=1}^{n-1} (x - x_j). \end{aligned}$$

Cada novo coeficiente acrescenta um termo ao polinômio, preservando os coeficientes anteriormente calculados.

Note que:

Para n pontos distintos, o polinômio interpolador possui grau máximo $n - 1$. A formulação incremental permite incorporar novos pontos sem recalcular os coeficientes anteriores.

MN Aplicados à Engenharia

Polinômio de Newton – Implementação em Python

Exercício resolvido

Um ensaio experimental forneceu os seguintes valores para a relação entre uma variável independente x e uma variável dependente y:

x_i	0	1	2	3
y_i	1	3	2	5

Determine o polinômio interpolador de Newton e estime o valor de $f(1.5)$.

Implementação em Python

```
import numpy as np
x = np.array([0, 1, 2, 3], dtype=float)
y = np.array([1, 3, 2, 5], dtype=float)
def newton(x, y, valor):
    n = len(x)
    a = y.copy()
    # Calculo das diferencas divididas
    for j in range(1, n):
        for i in range(n-1, j-1, -1):
            a[i] = (a[i] - a[i-1]) / (x[i] - x[i-j])
    # Avaliacao do polinomio de Newton
    resultado = a[n-1]
    for i in range(n-2, -1, -1):
        resultado = a[i] + (valor - x[i])*resultado
    return resultado, a
resultado, coef = newton(x, y, 1.5)
print("Coeficientes:", coef)
print(f"f(1.5) = {resultado:.4f}")
```

Resultado

$$\boxed{f(1.5) = 2.4375}$$

Coeficientes

$$a_1 = 1, a_2 = 2, a_3 = -1.5, a_4 = 1.1667$$

Exercícios de Polinômio de Newton

Implemente o polinômio interpolador de Newton nos exercícios do Polinômio de Lagrange.

1. Exercício I - slide 47.
2. Exercício II - slide 48.

Atenção

Implemente o polinômio de Newton e compare os resultados deste com os resultados do polinômio de Lagrange. Utilize métricas de erro: MAE e RMSE.

Organização

1. Ajustes de Curvas
   - Conceitos
2. Regressão Linear - mínimos quadrados
   - Implementação computacional
3. Análise de erros
   - Métricas de análise
4. Linearização
   - Conceitos
   - Implementação computacional
5. Regressão Polinomial
   - Implementação computacional
6. Polinômio interpolador de Lagrange
   - Implementação computacional
7. Polinômio interpolador de Newton
   - Implementação computacional
8. Splines: introdução
   - Splines Linear
     - Implementação computacional
   - Splines Quadráticas
     - Implementação computacional
   - Splines Cúbicas
     - Implementação computacional

MN Aplicados à Engenharia

Ajuste de Curvas e Interpolação

Splines: introdução

1. Na interpolação polinomial, um conjunto de $n + 1$ pontos distintos pode ser interpolado por um único polinômio de grau máximo n, que reproduz exatamente os valores conhecidos nos pontos amostrados.

2. Embora essa abordagem forneça uma solução exata nos pontos de interpolação, o aumento do grau do polinômio pode provocar oscilações indesejáveis entre os pontos, especialmente em conjuntos de dados com muitos pontos ou com distribuição inadequada dos nós.

3. Esse comportamento pode comprometer a representação da tendência dos dados e produzir estimativas pouco realistas, mesmo quando os valores interpolados nos nós são exatos.

4. Uma alternativa consiste na utilização de **funções spline**, que dividem o intervalo de interpolação em subintervalos e utilizam polinômios de graus menores, conectados por condições de continuidade e suavidade.

MN Aplicados à Engenharia

Ajuste de Curvas e Interpolação

Splines – Introdução

Para um mesmo conjunto de dados, é mais adequado um único polinômio ou vários polinômios?

MN Aplicados à Engenharia

Ajuste de Curvas e Interpolação

Splines – Introdução

Para um mesmo conjunto de dados, é mais adequado um único polinômio ou vários polinômios?

1. Para conjuntos de dados com **muitos pontos**, pode ser mais adequado utilizar vários polinômios de baixa ordem em vez de um único polinômio de grau elevado.
2. Cada polinômio é definido em um subintervalo do domínio, podendo representar o comportamento local dos dados experimentais.
3. Os polinômios possuem, em geral, o mesmo grau, mas apresentam coeficientes distintos em cada subintervalo.
4. A interpolação construída dessa maneira é denominada **interpolação polinomial por partes**, sendo conhecida como *interpolação por splines*.
5. Os pontos que delimitam os subintervalos e conectam os polinômios adjacentes são denominados **nós de interpolação**.

Tipos de splines

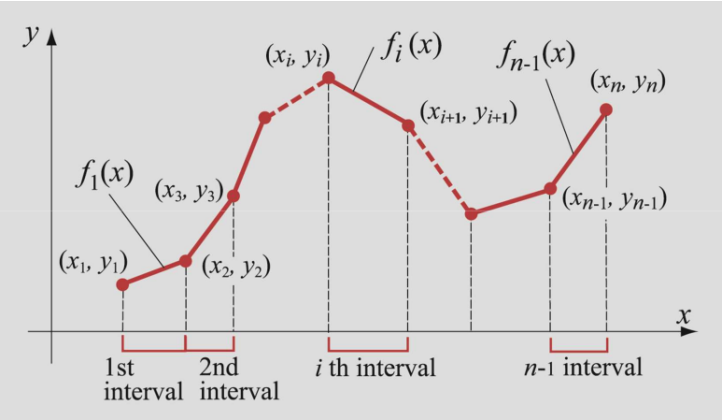
- Spline linear (polinômios de grau 1);
- Spline quadrática (polinômios de grau 2);
- Spline cúbica (polinômios de grau 3).

MN aplicados à Engenharia

Splines Linear

Considere a Figura 7. Perceba o perfil das funções f_i.

Figura 7: Spline Linear



Fonte: GILAT,(2008)

MN Aplicados à Engenharia

Splines Lineares

- A *spline* linear é uma técnica de interpolação polinomial por partes, na qual cada par de pontos consecutivos é conectado por um segmento de reta;
- Para construir a interpolação, utiliza-se um polinômio de primeiro grau em cada intervalo. Considerando os dois primeiros pontos, (x_1, y_1) e (x_2, y_2), o polinômio interpolador de **Lagrange** é dado por:

$$p(x) = s_1(x) = \frac{x - x_2}{x_1 - x_2} y_1 + \frac{x - x_1}{x_2 - x_1} y_2.$$

- De maneira geral, para um intervalo $[x_i, x_{i+1}]$, com $i = 1, 2, \ldots, n-1$, a interpolação é realizada pelo polinômio linear que conecta os pontos (x_i, y_i) e (x_{i+1}, y_{i+1}):

$$s_i(x) = \frac{x - x_{i+1}}{x_i - x_{i+1}} y_i + \frac{x - x_i}{x_{i+1} - x_i} y_{i+1},$$

- Portanto, a função interpoladora é definida por partes, sendo válida em cada intervalo:

$$\boxed{p(x) = s_i(x), \quad x_i \leq x \leq x_{i+1}}$$

MN aplicados à Engenharia

Exemplo de spline

Determine as *splines* lineares que fazem o ajuste dos dados: $x = [8, 11, 15, 18, 22]; y = [5, 9, 10, 8, 7]$.

- **Os cinco pontos geram quatro splines, portanto:**
  - Para $8 \leq x \leq 11$
    $$f_1(x) = \frac{(x - x_2)}{(x_1 - x_2)} y_1 + \frac{(x - x_1)}{(x_2 - x_1)} y_2 = \frac{(x - 11)}{(8 - 11)} 5 + \frac{(x - 8)}{(11 - 8)} 9 = \frac{5}{-3}(x - 11) + \frac{9}{2}(x - 8)$$
  - Para $11 \leq x \leq 15$
    $$f_2(x) = \frac{(x - x_3)}{(x_2 - x_3)} y_2 + \frac{(x - x_2)}{(x_3 - x_2)} y_3 = \frac{(x - 15)}{(11 - 15)} 9 + \frac{(x - 11)}{(15 - 11)} 10 = \frac{9}{-4}(x - 15) + \frac{10}{4}(x - 11)$$
  - Para $15 \leq x \leq 18$
    $$f_3(x) = \frac{(x - x_4)}{(x_3 - x_4)} y_3 + \frac{(x - x_3)}{(x_4 - x_3)} y_4 = \frac{(x - 18)}{(15 - 18)} 10 + \frac{(x - 15)}{(18 - 15)} 8 = \frac{10}{-3}(x - 18) + \frac{8}{3}(x - 15)$$
  - Para $18 \leq x \leq 22$
    $$f_4(x) = \frac{(x - x_5)}{(x_4 - x_5)} y_4 + \frac{(x - x_4)}{(x_5 - x_4)} y_5 = \frac{(x - 22)}{(18 - 22)} 8 + \frac{(x - 18)}{(22 - 18)} 7 = -2(x - 18) + \frac{7}{4}(x - 22)$$

- O **valor interpolado de y em $x = 12,7$ é obtido com a substituição do valor x na equação de $f_2(x)$ acima resultando em:**

$$f_2(x) = \frac{9}{-4}(x - 15) + \frac{10}{4}(x - 11) = \frac{9}{-4}(12,7 - 15) + \frac{10}{4}(12,75 - 11) = \mathbf{9,425}$$

MN Aplicados à Engenharia

Exemplo de Spline Linear com Python

1. Construção das splines

```
import numpy as np
import matplotlib.pyplot as plt
# Dados experimentais
x = np.array([8, 11, 15, 18, 22])
y = np.array([5, 9, 10, 8, 7])
# Splines lineares
for i in range(len(x)-1):
    a = (y[i+1]-y[i])/(x[i+1]-x[i])
    b = y[i] - a*x[i]
    print(f"Spline {i+1}:")
    print(f"Intervalo: [{x[i]},
    {x[i+1]}]")
    print(f"f{i+1}(x) = {a:.4f}x +
    {b:.4f}")
# Interpolacao
xi = 12.7
yi = np.interp(xi, x, y)
print(f"\nValor interpolado:")
print(f"f({xi}) = {yi:.3f}")
```

2. Representação gráfica

```
xx = np.linspace(x[0], x[-1], 300)
yy = np.interp(xx, x, y)
plt.plot(x, y, 'ro',
label='Dados experimentais')
plt.plot(xx, yy, 'b-',label='Spline linear')
plt.plot(xi, yi, 'ks',
label='Valor interpolado')
plt.xlabel('x'); plt.ylabel('y')
plt.title('Splines Lineares')
plt.grid(); plt.legend()
plt.tight_layout()
plt.show()
```

Resultado da interpolação: $\boxed{f(12,7) = 9,425}$

Equações obtidas:

$$f_1(x) = 1,3333x - 5,6667$$
$$f_2(x) = 0,25x + 6,25$$
$$f_3(x) = -0,6667x + 20$$
$$f_4(x) = -0,25x + 12,5$$

MN Aplicados à Engenharia

Ajuste de Curvas e Interpolação

Limitações das Splines Lineares

1. Embora contínua, a função interpoladora por *splines* lineares geralmente não possui derivada primeira contínua nos pontos de conexão entre os intervalos;
   - Cada segmento possui inclinação constante, que pode diferir da inclinação dos segmentos adjacentes, provocando mudanças abruptas na derivada;
2. Em aplicações de Engenharia, essa falta de suavidade pode limitar a representação de fenômenos físicos que apresentam transições suaves;
3. Uma alternativa são as *splines* cúbicas, que garantem continuidade da função e de suas duas primeiras derivadas (C^2).

MN Aplicados à Engenharia

Exercícios de Fixação – Splines Lineares com Python

1. **Implementação de uma Spline Linear**
Considere os pontos experimentais:

$$x = [0, 1, 2, 3, 4], \qquad y = [1, 2, 1, 3, 2].$$

- Implemente a interpolação utilizando `interp1d`, com interpolação linear;
- Construa o gráfico dos pontos e da função interpoladora.

2. **Avaliação da Função Interpoladora**
Considere os dados:

$$x = [20, 22, 24, 26], \qquad y = [1, 5, 3, 7].$$

- Construa a spline linear;
- Determine os valores interpolados em $x = 1,\ 3,\ 5$;
- Calcule e compare as inclinações dos segmentos.

3. **Aplicação a Dados de Engenharia**
A tabela representa a temperatura de um sistema em diferentes instantes:

t (s)	10	12	14	16	18
T ($^\circ$C)	25	32	38	35	30

- Implemente a spline linear para representar a evolução da temperatura;
- Estime a temperatura em $t = 1,\ 3,\ 5,\ 7$ s;
- Analise as mudanças de inclinação nos pontos de interpolação.

MN aplicados à Engenharia

Splines Quadráticas

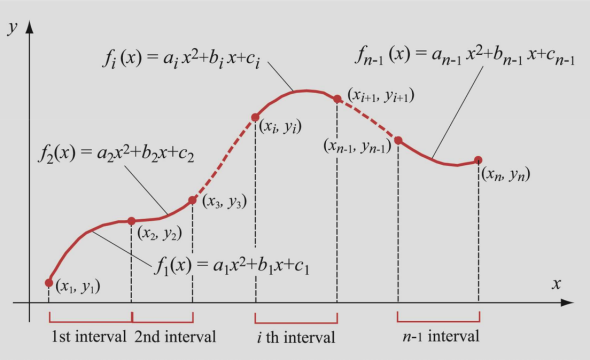
Considere a Figura 8.

- As *splines* quadráticas são funções de interpolação construídas a partir de polinômios de segundo grau, definidos individualmente em cada intervalo entre dois pontos consecutivos;
- Para um conjunto de n pontos distintos, $(x_1, y_1), \ldots, (x_n, y_n)$, são definidos $n-1$ intervalos de interpolação;
- Em cada intervalo $[x_i, x_{i+1}]$, a função interpoladora é representada por um polinômio quadrático:

$$S_i(x) = a_i x^2 + b_i x + c_i, \quad i = 1, 2, \ldots, n-1.$$

- Os polinômios são determinados de modo que a função interpoladora passe por todos os pontos fornecidos, garantindo a continuidade da curva nos intervalos adjacentes.

Figura 8: Interpolação por splines quadráticas



Fonte: Adaptado de Gilat (2008).

MN aplicados à Engenharia

Splines Quadráticas

- Para um conjunto de n pontos, são definidos $n-1$ intervalos de interpolação. Em cada intervalo, utiliza-se um polinômio de segundo grau:

$$S_i(x) = a_i x^2 + b_i x + c_i.$$

- Uma vez que cada polinômio possui três coeficientes, o número total de incógnitas é: $N_c = 3(n-1) = 3n-3$.
- Para determinar os coeficientes, são estabelecidas as seguintes condições:

1. **Interpolação dos pontos:**
Cada polinômio deve passar pelos dois pontos extremos de seu intervalo, garantindo:

$$\begin{cases} S_i(x_i) = y_i \\ S_i(x_{i+1}) = y_{i+1} \end{cases}$$

Substituindo o polinômio, obtêm-se:

$$\begin{cases} a_i x_i^2 + b_i x_i + c_i = y_i \\ a_i x_{i+1}^2 + b_i x_{i+1} + c_i = y_{i+1} \end{cases}$$

2. **Continuidade da primeira derivada:**
Nos pontos internos, a inclinação dos polinômios adjacentes deve ser igual, evitando descontinuidades na tangente à curva, de modo que:

$$S_i'(x_{i+1}) = S_{i+1}'(x_{i+1}), \quad i = 1, \ldots, n-2.$$

Uma vez: $S_i'(x) = 2a_i x + b_i$, $\Rightarrow 2a_i x_{i+1} + b_i = 2a_{i+1} x_{i+1} + b_{i+1}$.

MN aplicados à Engenharia

Splines Quadráticas – Sistema de Equações

- A aplicação das condições anteriores fornece:
  - Interpolação: $2(n-1)$ equações;
  - Continuidade da primeira derivada: $n-2$ equações.
- Portanto, o número total de equações é:

$$N_e = 2(n-1) + (n-2) = 3n - 4.$$

- Entretanto, existem $3n-3$ incógnitas. Assim, o sistema apresenta um grau de liberdade, sendo necessária uma condição adicional.

3. **Condição de contorno:** Uma possibilidade é estabelecer que o primeiro polinômio seja linear:

$$\boxed{a_1 = 0}$$

- Com essa condição adicional, o sistema passa a apresentar o mesmo número de equações e incógnitas, permitindo determinar os coeficientes das splines quadráticas.

MN aplicados à Engenharia

Splines Quadráticas – Continuidade e Sistema

Continuidade da primeira derivada

Em cada nó interno, a inclinação dos polinômios adjacentes deve ser igual. Essa condição garante uma transição suave entre os segmentos da curva.

- A derivada primeira do polinômio no intervalo $[x_i, x_{i+1}]$ é: $S_i'(x) = 2a_i x + b_i$.
- Igualando as derivadas nos nós internos, obtém-se: $2a_{i-1}x_i + b_{i-1} = 2a_i x_i + b_i \quad para i = 2, \ldots, n-1$.
- Como existem $n-2$ nós internos, essa condição fornece $n-2$ equações adicionais.

Contagem das equações e incógnitas

Equações

$$N_e = 2(n-1) + (n-2)$$

$$\boxed{N_e = 3n - 4}$$

Incógnitas

$$N_c = 3(n-1)$$

$$\boxed{N_c = 3n - 3}$$

Conclusão

Existe uma equação a menos que o número de incógnitas. Portanto, é necessária uma condição de contorno adicional para determinar unicamente os coeficientes.

MN aplicados à Engenharia

Splines Quadráticas – Condição de Contorno

- Para que o sistema linear tenha solução única, é necessário estabelecer uma condição adicional de contorno.
- Uma escolha usual consiste em impor que a derivada segunda seja nula em uma das extremidades do domínio. Portanto, para $x_1 \to S_1''(x_1) = 0$.
- Considerando o primeiro polinômio: $S_1(x) = a_1x^2 + b_1x + c_1$, sua derivada segunda é: $S_1''(x) = 2a_1$.
- Aplicando a condição de contorno: $2a_1 = 0 \implies \boxed{a_1 = 0}$.

Interpretação geométrica

Como $a_1 = 0$, o primeiro segmento é representado por uma função linear:

$$S_1(x) = b_1x + c_1.$$

Portanto, uma reta conecta os dois primeiros pontos interpolados.

Sistema completo

Com a condição adicional, são obtidas $3n - 3$ equações para determinar os $3n - 3$ coeficientes das splines.

MN aplicados à Engenharia

Splines Quadráticas – Exemplo Numérico

Problema

Determine as splines quadráticas que interpolam os dados apresentados a seguir e calcule o valor de $S(12{,}7)$.

$$x = [8,\ 11,\ 15,\ 18,\ 22],$$
$$y = [5,\ 9,\ 10,\ 8,\ 7].$$

- Os cinco pontos definem quatro intervalos e, consequentemente, quatro polinômios quadráticos:

$$S_i(x) = a_i x^2 + b_i x + c_i, \quad i = 1, 2, 3, 4.$$

- Cada polinômio deve interpolar os dois pontos extremos de seu intervalo, totalizando oito equações.

Primeiros intervalos

$$S_1(8) = 5$$
$$S_1(11) = 9$$
$$S_2(11) = 9$$
$$S_2(15) = 10$$

Últimos intervalos

$$S_3(15) = 10$$
$$S_3(18) = 8$$
$$S_4(18) = 8$$
$$S_4(22) = 7$$

Primeira condição de contorno

Para completar a formulação, adotamos: $\boxed{a_1 = 0}$

MN aplicados à Engenharia

Splines Quadráticas – Montagem do Sistema Linear

Continuidade da primeira derivada

Nos três nós internos, $x_2 = 11$, $x_3 = 15$ e $x_4 = 18$, as derivadas dos polinômios adjacentes devem ser iguais:

$$i = 2: \quad 22a_1 + b_1 = 22a_2 + b_2$$

$$i = 3: \quad 30a_2 + b_2 = 30a_3 + b_3$$

$$i = 4: \quad 36a_3 + b_3 = 36a_4 + b_4$$

Contagem das equações

$$\underbrace{8}_{\text{Interpolação}} + \underbrace{3}_{\text{Continuidade}} = \boxed{11 \text{ equações}}$$

Determinação dos coeficientes

Com a condição de contorno $a_1 = 0$, obtém-se um sistema de 11 equações e 11 incógnitas, cuja solução determina os coeficientes restantes.

Representação matricial

O sistema linear é expresso por:

$$\boxed{\mathbf{Ac} = \mathbf{b}}$$

em que o vetor de coeficientes é:

$$\mathbf{c} = \begin{bmatrix} a_1 \\ b_1 \\ c_1 \\ a_2 \\ b_2 \\ c_2 \\ a_3 \\ b_3 \\ c_3 \\ a_4 \\ b_4 \\ c_4 \end{bmatrix}$$

$$\mathbf{A} \in \mathbb{R}^{11\times 12}, \qquad \mathbf{c} \in \mathbb{R}^{12}.$$

MN aplicados à Engenharia

Splines Quadráticas – Representação Matricial

- A matriz **A** reúne os coeficientes das equações de interpolação e continuidade, enquanto **b** contém os valores conhecidos dos pontos.
- O vetor **c** organiza os coeficientes dos quatro polinômios, na sequência:

$$\mathbf{c} = [a_1, b_1, c_1,\ a_2, b_2, c_2,\ a_3, b_3, c_3,\ a_4, b_4, c_4]^T.$$

- Conhecendo-se $a_1 = 0$, determina-se o vetor dos onze coeficientes restantes por meio da solução do sistema linear.

MN aplicados à Engenharia

Splines Quadráticas – Solução e Interpolação

Coeficientes determinados

A solução do sistema linear fornece o seguinte vetor de coeficientes:

$$\mathbf{c}^T = [\underbrace{0{,}0000,\ 1{,}3333,\ -5{,}6667}_{S_1}\ \underbrace{-0{,}2708,\ 7{,}2917,\ -38{,}4375}_{S_2}\ \underbrace{0{,}0556,\ -2{,}5000,\ 35{,}0000}_{S_3}\ \underbrace{0{,}0625,\ -2{,}7500,\ 37{,}2500}_{S_4}].$$

Determinação de $S(12{,}7)$

- Como $12{,}7 \in [11, 15]$, a avaliação deve utilizar o segundo polinômio, $S_2(x)$: $S_2(x) = -0{,}2708333x^2 + 7{,}2916667x - 38{,}4375$.
- Substituindo $x = 12{,}7$: $S_2(12{,}7) = -0{,}2708333(12{,}7)^2 + 7{,}2916667(12{,}7) - 38{,}4375$.

Resultado da interpolação

$$\boxed{S(12{,}7) \approx 10{,}48396}$$

MN aplicados à Engenharia

Splines Quadráticas – Solução Computacional (I)

Implementação em Python

O sistema linear é construído a partir das condições de interpolação, continuidade da primeira derivada e da condição de contorno $a_1 = 0$.

```
import numpy as np
# Dados de interpolacao
x = np.array([8, 11, 15, 18, 22], dtype=float)
y = np.array([5, 9, 10, 8, 7], dtype=float)
n = len(x)
m = n - 1
# Sistema linear A*c = b
A, b = [], []
# 1. Condicoes de interpolacao
for i in range(m):
    for k in [i, i + 1]:
        linha = np.zeros(3*m)
        linha[3*i:3*i+3] = [x[k]**2, x[k], 1]
        A.append(linha)
        b.append(y[k])
# 2. Continuidade da primeira derivada
for i in range(1, m):
    xi = x[i]
    linha = np.zeros(3*m)
    linha[3*(i-1):3*(i-1)+2] = [2*xi, 1]
    linha[3*i:3*i+2] = [-2*xi, -1]
    A.append(linha)
    b.append(0)
# 3. Condicao de contorno: a1 = 0
linha = np.zeros(3*m)
linha[0] = 1
A.append(linha)
b.append(0)
# Resolucao do sistema
A = np.array(A)
b = np.array(b)
c = np.linalg.solve(A, b)
# Organizacao dos coeficientes
coef = c.reshape(m, 3)
print("Coeficientes das splines:")
print(coef)
```

Resultado da implementação

O sistema possui 12 equações e 12 incógnitas, determinando os coeficientes dos quatro polinômios.

MN aplicados à Engenharia

Splines Quadráticas – Solução Computacional (II)

Avaliação em $x = 12{,}7$

O ponto pertence ao segundo intervalo, pois $11 \leq 12{,}7 \leq 15$. Portanto, utiliza-se $S_2(x)$.

```
        # Exibicao dos polinomios
for i, (a, b, c) in enumerate(coef, start=1):
    print(f"S{i}(x) = {a:.6f}x^2 "
    f"+ ({b:.6f})x + ({c:.6f})")

# Avaliacao da spline em x = 12.7
x_eval = 12.7

# Identificacao do intervalo
i = np.searchsorted(x, x_eval, side="right") - 1
i = min(i, m - 1)

# Avaliacao do polinomio correspondente
a, b, c = coef[i]
resultado = a*x_eval**2 + b*x_eval + c

print(f"\nIntervalo utilizado: [{x[i]}, {x[i+1]}]")
print(f"S({x_eval}) = {resultado:.6f}")
```

Coeficientes calculados

$$S_1(x) = 1{,}333333x - 5{,}666667$$

$$S_2(x) = -0{,}270833x^2 + 7{,}291667x - 38{,}437500$$

$$S_3(x) = 0{,}055556x^2 - 2{,}5x + 35$$

$$S_4(x) = 0{,}062500x^2 - 2{,}75x + 37{,}25$$

Resultado final

$$\boxed{S(12{,}7) = S_2(12{,}7) \approx 10{,}483958}$$

MN aplicados à Engenharia

Splines Quadráticas – Exercícios de Fixação

Exercício 1 – Construção das splines

Considere os pontos:

$$x = [0, 5, 10, 15, 20, 25, 30, 35], \qquad y = [25, 55, 90, 120, 135, 130, 110, 85].$$

Implemente em Python a construção do sistema linear para determinar as três splines quadráticas, adotando a condição de contorno $a_1 = 0$.

Exercício 2 – Avaliação da interpolação

Utilizando os dados do Exercício 1, determine o valor interpolado em $x = 3{,}5, 22.25, 34.15$. Identifique automaticamente o intervalo correspondente e selecione o polinômio adequado para calcular $S(3{,}5)$.

Exercício 3 – Verificação da continuidade

Com os coeficientes obtidos no Exercício 1, implemente uma rotina para verificar a continuidade da primeira derivada nos nós internos $x = 10$ e $x = 25$, avaliando:

$$S_i'(x_{i+1}) = S_{i+1}'(x_{i+1}).$$

Orientações

Apresente o código em Python, os coeficientes calculados, os resultados numéricos e a interpretação dos resultados obtidos.

MN aplicados à Engenharia

Interpolação por Splines Cúbicas

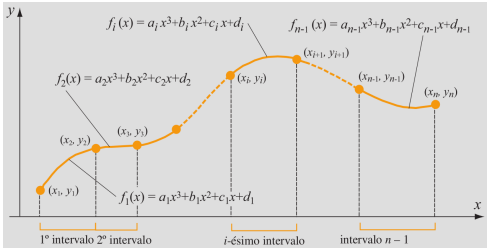
Considere a Figura 9.

- As *splines* cúbicas utilizam polinômios de terceiro grau para interpolar conjuntos de dados definidos em intervalos consecutivos.
- Para um conjunto de n pontos distintos, são definidos $n-1$ intervalos, cada um associado a um polinômio cúbico:

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i, \texttt{ em que } i = 1, \ldots, n-1.$$

- Cada polinômio possui quatro coeficientes desconhecidos. Portanto, são necessários $4(n-1)$ coeficientes para representar o conjunto de splines.
- A construção exige condições de interpolação, continuidade e condições de contorno para determinar unicamente os polinômios.

Figura 9: Spline Cúbica



Fonte: Gilat (2008).

Condições de continuidade

$$S_i(x_{i+1}) = S_{i+1}(x_{i+1})$$
$$S_i'(x_{i+1}) = S_{i+1}'(x_{i+1})$$
$$S_i''(x_{i+1}) = S_{i+1}''(x_{i+1})$$

Continuidade de ordem C^2.

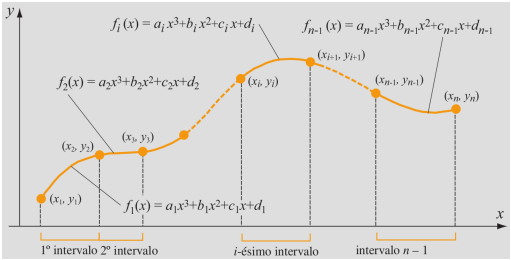
MN aplicados à Engenharia

(1) De forma geral, há $n-1$ equações e, como cada equação tem, *neste caso*, **quatro** coeficientes, um total de $4(n-1) = 4n-4$ coeficientes a serem determinados, tais que:

* Cada polinômio $f_i(x)$ deve passar pelos pontos finais do intervalo, (x_i, y_i) e (x_{i+1}, y_{i+1}), o que significa que $f_i(x_i) = y_i$ e $f_i(x_{i+1}) = y_{i+1}$, portanto:

$$\begin{cases} a_i x_i^3 + b_i x_i^2 + c_i x_i + d_i = y_i \\ a_i x_{i+1}^3 + b_i x_{i+1}^2 + c_i x_{i+1} + d_{i+1} = y_{i+1} \end{cases} \quad \text{para } i = 1, 2, \ldots, n-1$$

Figura 10: Splines Quadráticas



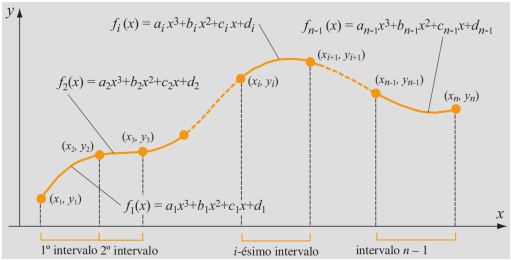
Fonte: GILAT,(2008)

MN aplicados à Engenharia

(2) Nos nós internos, as inclinações (derivadas primeiras) dos polinômios de intervalos adjacentes são iguais, portanto, com a transição da curva que passa por um nó interno de um polinômio para outro:

- A derivada primeira do i-ésimo polinômio é: $f_i'(x) = \dfrac{df_i}{dx} = 3a_i x^2 + 2b_i x + c_i$
- Para n pontos, o primeiro ponto interno é $i = 2$ e o último é $i = n - 1$, o que resulta em: $3a_{i-1}x_i^2 + 2b_{i-1}x_i + c_i = 3a_i x_i^2 + 2b_i x_i + c_i$
- Como há $n - 2$ pontos internos, essa condição fornece $n - 2$ equações.

Figura 11: Splines Quadráticas



Fonte: GILAT,(2008)

MN aplicados à Engenharia

(3) Nos nós internos, as derivadas segundas dos polinômios de intervalos adjacentes devem ser iguais, isso significa que, com a transição da curva que passa por um nó interno de um polinômio para outro, a taxa de inclinação (curvatura) deve ser contínua, portanto,:

- $f_i''(x) = \dfrac{d^2 f_i}{dx^2} = 6a_i x + 2b_i$
- Para n pontos, o primeiro ponto interno é $i = 2$ e o último é $i = n - 1$, o que resulta em: $6a_{i-1}x_i + 2b_{i-1} = 6a_i x_i + 2b_i$
- Como há $n - 2$ pontos internos, essa condição fornece $n - 2$ equações.

- Juntas, as três condições fornecem $4n-6$ equações, entretanto, os $n-1$ *polinômios* têm $4n-4$ *coeficientes*, e com isso duas equações (condições) adicionais são necessárias para que os coeficientes sejam obtidos. As condições geralmente escolhidas assumem que **a derivada segunda seja nula no primeiro e no último ponto**, portanto:
  $6a_1x_1 + 2b_1 = 0$ e $6a_{n-1}x_n + 2b_{n-1} = 0$

* Splines cúbicas com derivadas segundas igualadas a zero nos pontos finais do intervalo são chamadas de **splines cúbicas naturais**.

- A aplicação de todas as condições leva a um sistema de $4n - 4$ **equações com** $4n - 4$ **coeficientes**.

MN aplicados à Engenharia

Splines Cúbicas
Baseadas em polinômios na forma de Lagrange

- A dedução de splines cúbicas utilizando a forma de Lagrange resulta no termo geral:

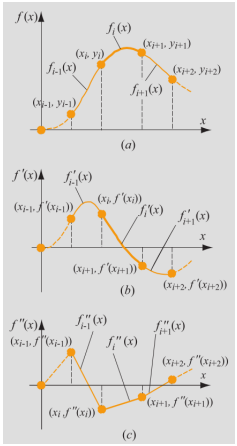
$$f_i(x) = \begin{cases} \frac{a_i}{6h_i}(x_{i+1} - x)^3 + \frac{a_{i+1}}{6h_i}(x - x_i)^3 \\ \quad + \left[\frac{y_i}{h_i} - \frac{a_i h_i}{6}\right](x_{i+1} - x) \\ \quad + \left[\frac{y_{i+1}}{h_i} - \frac{a_{i+1} h_i}{6}\right](x - x_i), \end{cases} ,$$

em que $x_i \leq x \leq x_{i+1}$ e $i = 1, 2, \ldots, n-1$

- E o termo a_i vale:

$$h_i a_i + 2(h_i + h_{i+1})a_{i+1} + h_{i+1}a_{i+2} = 6\left[\frac{y_{i+2} - y_{i+1}}{h_{i+1}} - \frac{y_{i+1} - y_i}{h_i}\right], \text{ para } i = 1, 2, \ldots, n-1.$$

Figura 12: Splines Cúbica



Fonte: GILAT,(2008)

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Exemplo: Determine as *splines* cúbicas que fazem o ajuste dos dados: $x = [8, 11, 15, 18, 22]; y = [5, 9, 10, 8, 7]$, e encontre $f(12, 7)$.

- **Os cinco pontos geram quatro splines**. A equação cúbica da i-ésima spline é:
  $f_i(x) = \frac{a_i}{6h_i}(x_{i+1} - x)^3 + \frac{a_{i+1}}{6h_i}(x - x_i)^3 + \left[\frac{y_i}{h_i} - \frac{a_i h_i}{6}\right](x_{i+1} - x) + \left[\frac{y_{i+1}}{h_i} - \frac{a_{i+1} h_i}{6}\right](x - x_i)$, para $i = 1, \ldots 4$, em que $h_i = x_{x+1} - x_i$
- As quatro equações contêm cinco coeficientes desconhecidos a_1, a_2, a_3, a_4 e a_5. Nas splines cúbicas naturais, os coeficientes a_1 e a_5 são iguais a zero.
- Os outros três coeficientes são determinados a partir de um sistema linear de três equações, a saber:
  - $h_1 = x_2 - x_1 = 11 - 8 = 3$ e $h_2 = x_3 - x_2 = 15 - 11 = 4$;
  - $h_3 = x_4 - x_3 = 18 - 15 = 3$ e $h_4 = x_5 - x_4 = 22 - 18 = 4$.

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Splines cúbicas - Exemplo

De modo que tem-se para:

i=1 $h_1 a_1 + 2(h_1 + h_2)a_2 + h_2 a_3 = 6\left[\frac{y_3 - y_2}{h_2} - \frac{y_2 - y_1}{h_1}\right]$

$3.\boxed{0} + 2(3+4)a_2 + 4a_3 = 6\left[\frac{10-9}{4} - \frac{9-5}{3}\right] \rightarrow 14a_2 + 4a_3 = -6,5$

i=2 $h_2 a_2 + 2(h_2 + h_3)a_3 + h_3 a_4 = 6\left[\frac{y_4 - y_3}{h_3} - \frac{y_3 - y_2}{h_2}\right]$

$4.a_2 + 2(3+4)a_3 + 3a_4 = 6\left[\frac{8-10}{3} - \frac{10-9}{4}\right] \rightarrow 4a_2 + 14a_3 + 3a_4 = -5,5$

i=3 $h_3 a_3 + 2(h_3 + h_4)a_4 + h_4 a_5 = 6\left[\frac{y_5 - y_4}{h_4} - \frac{y_4 - y_3}{h_3}\right]$

$3.a_3 + 2(3+4)a_4 + 4.\boxed{0} = 6\left[\frac{7-8}{4} - \frac{8-10}{3}\right] \rightarrow 3a_3 + 14a_4 = 2,5$

- Na forma matricial o sistema resulta em: $\begin{bmatrix} 14 & 4 & 0 \\ 4 & 14 & 3 \\ 0 & 3 & 14 \end{bmatrix} \begin{bmatrix} a_2 \\ a_3 \\ a_4 \end{bmatrix} = \begin{bmatrix} -6,5 \\ -5,5 \\ 2,5 \end{bmatrix}$

Portanto:

$a_1 = 0$; $a_2 = -0,3665$; $a_3 = -0,3421$; $a_4 = 0,2519$ e $a_5 = 0$

MN aplicados à Engenharia

De modo que :

- O polinômio interpolado $p(x) = f(x)$ calculado com a substituição dos coeficientes para o intervalo que contém $x = 12,7$ é $f_2(x)$ com $11 \leq x \leq 15$

i=1 ...

i=2 $f_2(x) = \frac{a_2}{6h_2}(x_3 - x)^3 + \frac{a_3}{6h_2}(x - x_2)^3 + \left[\frac{y_2}{h_2} - \frac{a_2 h_2}{6}\right](x_3 - x) + \left[\frac{y_3}{h_2} - \frac{a_3 h_2}{6}\right](x - x_2)$

$f_2(x) = \frac{-0,3665}{6.4}(15 - 3)^3 + \frac{-0,3421}{6.4}(x - 11)^3 + \left[\frac{9}{4} - \frac{-0,3665.4}{6}\right](15 - x) + \left[\frac{10}{4} - \frac{-0,3421.4}{6}\right](x - 11)$

$f_2(x) = (-0,01527)(15 - x)^3 + (-0,01427)(x - 11)^3 + 2,494(15 - x) + 2,728(x - 11)$

i=3 ...

i=4 ...

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Portanto,

- $f_2(x) = (-0,01527)(15 - x)^3 + (-0,01427)(x - 11)^3 + 2,494(15 - x) + 2,728(x - 11)$
- $f_2(12,7) = (-0,01527)(15 - 12,7)^3 + (-0,01427)(12,7 - 11)^3 + 2,494(15 - 12,7) + 2,728(12,7 - 11) =$ **10,11**

Comparativo de resultados das splines para $x_{int} = 12,7$

$x = 12,7$	Linear	Quadrática	Cúbica
$f(12,7)$	$9,4250$	$10,4898$	$10,11$

MN aplicados à Engenharia

Ajuste de Curvas e interpolação

Funções auxiliares nativas

- polyfit(*vetor x, vetor y , grau*)
- interp1(*vetor x, vetor y, x interpolado (vetor ou escalar), método*)
  - Interpolação : 'nearest' e 'linear'
  - Interpolação e extrapolação: 'spline' e 'pchip'
  - 'spline' exige que o intervalos estejam equidistantes para mitigar erros.

Referências

- ÁVILA, Sérgio Luciano. **Cálculo numérico aplicado à engenharia elétrica com MATLAB**.
- COSTA, Jonatha R. **Notas de aula: Métodos Numéricos**. Disponível em: `https://github.com/jonathacosta/NM`.
- COSTA, Jonatha R. **Códigos em Python**. Disponível em: `https://github.com/jonathacosta/NM`.
- GILAT, Amos. **MATLAB com aplicações em engenharia**. Bookman, 2009.
- IFCE. **PUD – Programa da Unidade Didática**. Bacharelado em Controle e Automação, *campus* Maracanaú.